

损失函数也会影响优化难度

李宏毅《机器学习》2025 · 类神经网络训练不起来怎么办
(四)



基于李宏毅老师课程整理

2026 年 6 月 8 日

视频信息

频道：李宏毅深度学习课堂 (Bilibili)

时长：约 20 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=7>

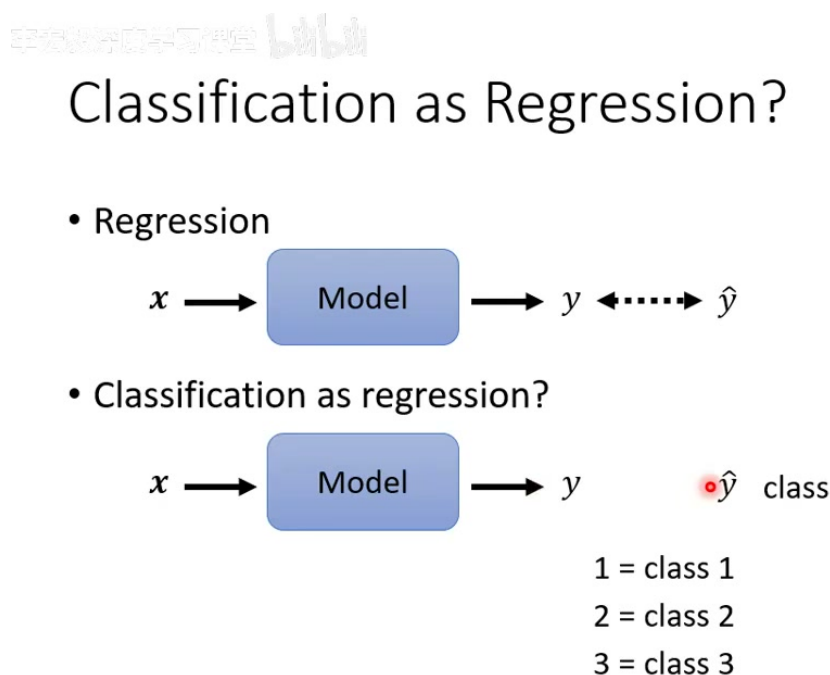
目录

1 引言：分类问题能不能直接当回归做？	3
2 One-hot 标签与多输出网络	4
3 Softmax：把 logits 变成可比较的分布	5
4 分类损失：MSE 与 Cross-entropy	8
4.1 从 likelihood 看 cross-entropy	9
5 为什么 Cross-entropy 更适合分类优化？	9
5.1 把梯度写出来：差别为什么会这么大？	11
6 本讲综合：loss function 也是优化设计的一部分	12

1 引言：分类问题能不能直接当回归做？

这一讲是分类问题的短版说明。前几讲已经建立了回归模型、梯度下降、批次、动量、自适应学习率等观念；本讲把焦点换成一个很实际的问题：**做分类时，输出、softmax 和 loss function 应该怎么搭配？**

最直接的想法是把分类也当成回归：输入 x ，模型输出一个数值 y ，然后把类别也编号成数字，例如 $1 = \text{class 1}$ 、 $2 = \text{class 2}$ 、 $3 = \text{class 3}$ ，再让 y 尽量接近正确类别编号。



3

图 1: 把分类当回归做的朴素想法：模型仍然输出单一标量 y ，再让 y 接近类别编号。但这等于把 class 1、class 2、class 3 放在一条有距离关系的数轴上。¹

类别编号会偷偷加入“相似度”假设

如果把 class 1 编成 1、class 2 编成 2、class 3 编成 3，模型会隐含地认为 class 1 比较接近 class 2，而比较远离 class 3。这个假设在某些有序类别中可能合理，例如年级、等级、评分；但在一般分类中通常不成立。例如猫、狗、车之间没有天然的一维顺序，把它们硬塞进 1, 2, 3 会给模型错误的几何结构。

这里的重点不是说“类别永远不能编号”，而是要看编号是否承载了真实语义。若任务本身是有序预测，例如一年级、二年级、三年级，编号有可能反映真实顺序；但若任务是手写数字、宝可梦种类、动物类别，类别编号往往只是资料集作者随手排的 ID。模型若把这个 ID 当成连续数值，就会学习到不存在的顺序关系。

因此，分类问题通常不用一个数字表示类别，而是把类别表示为 **one-hot vector**。

¹ 视频画面与字幕时间区间：01:25–02:08。

2 One-hot 标签与多输出网络

若共有三个类别，则正确标签 $\hat{y}$ 可以写成三维 one-hot vector：

$$\text{class 1: } \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \text{class 2: } \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \text{class 3: } \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

这种表示方法的好处是：每一类都占一个独立维度，类别之间不再因为编号大小产生人为远近。若用欧氏距离看，三个 one-hot 向量两两之间的距离相同，比较符合一般分类任务中“类别只是不同，不一定有顺序”的直觉。

One-hot 做了什么，又没有做什么

One-hot 的作用是把类别 ID 从连续数轴上解放出来。它告诉模型：这些类别是彼此不同的离散选项，而不是 $1 < 2 < 3$ 的连续量。但 one-hot 本身并不知道类别之间的语义相似性。例如 class 1 与 class 2 是否真的相近，one-hot 不表达；它只负责给 supervised learning 一个干净的目标格式。若未来想表达类别语义，那是 embedding、metric learning 或 label hierarchy 的问题，不是这讲要解决的分​​类基本设定。

Class as one-hot vector

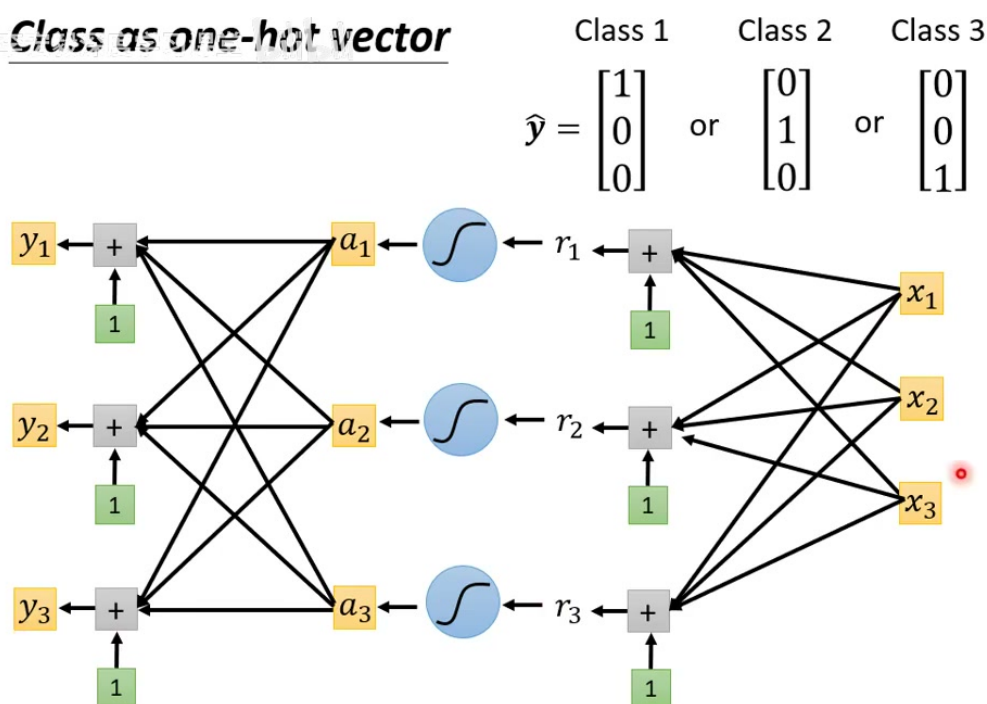


图 2：当标签变成三维 one-hot vector，网络也要输出三个数值 y_1, y_2, y_3 。做法并不神秘：把原本输出一个数值的最后一层改成输出多个数值，每个输出都有自己的一组权重和偏置。²

以一个隐藏层网络为例，回归任务可以写成

²视频画面与字幕时间区间：04:55–05:26。

$$y = b + c^T \sigma(b + Wx),$$

其中 y 是一个标量。分类时，最后一层输出不再是单一数字，而是向量：

$$\mathbf{y} = \mathbf{b}' + W' \sigma(\mathbf{b} + W\mathbf{x}) = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}.$$

这些 y_i 还不是概率，也不一定介于 0 与 1 之间。它们只是网络最后一层吐出的原始分数，常称为 **logits**。下一步，就是用 softmax 把 logits 变成可与 one-hot 标签比较的输出。

从标量输出到向量输出

分类网络的关键改变不是“网络变成另一种东西”，而是**最后一层的输出维度变成类别数**。若有 K 个类别，网络输出 K 个 logits；第 i 个 logit 可以理解成模型对第 i 类的未归一化支持程度。

这个改动也解释了为什么分类模型的最后一层常写成一个矩阵乘法。若隐藏层表示为 $\mathbf{h} = \sigma(\mathbf{b} + W\mathbf{x})$ ，最后一层就是

$$\mathbf{y} = W'\mathbf{h} + \mathbf{b}',$$

其中 W' 的每一行对应一个类别的打分器。换句话说，分类不是只训练一个回归器，而是同时训练 K 个类别打分器，再把这些分数交给 softmax 统一比较。

分类模型里的四个对象

一条标准多分类链路里，最好把下面四个对象分清楚：

- $\mathbf{x}$ ：输入特征，例如影像、语音特征、文字向量等。
- $\mathbf{y}$ ：网络输出的 logits，尚未归一化，可以是任意实数。
- $\mathbf{y}'$ 或 $\mathbf{p}$ ：softmax 后的输出，所有分量为正且总和为 1。
- $\hat{\mathbf{y}}$ 或 $\mathbf{q}$ ：正确答案的 one-hot 标签。

训练时真正比较的是 $\mathbf{p}$ 与 $\mathbf{q}$ ；但优化时真正被更新的是产生 logits $\mathbf{y}$ 的网络参数。

3 Softmax：把 logits 变成可比较的分布

one-hot 标签里的元素只会是 0 或 1，而 logits 可以是任意实数。为了让网络输出与标签在同一种尺度上比较，分类模型通常在 logits 后接一个 **softmax**：

Regression

$$\text{label } \hat{y} \longleftrightarrow y = b + c^T \sigma(b + Wx)$$

feature

Classification

$$y = b' + W' \sigma(b + Wx)$$

$$\text{label } \hat{y} \longleftrightarrow y' = \text{softmax}(y)$$

0 or 1 Can have any value

6

图 3: 分类模型的输出链路: 网络先产生 logits y , 再经过 softmax 得到 y' , 最后计算 y' 与 one-hot 标签 $\hat{y}$ 之间的误差。³

softmax 的定义为

$$y'_i = \frac{\exp(y_i)}{\sum_j \exp(y_j)}.$$

它做了两件重要的事:

- 先用 exponential 把每个 logit 变成正数, 因此 $y'_i > 0$ 。
- 再除以所有 exponential 的和, 因此 $\sum_i y'_i = 1$ 。

所以 y' 可以被理解为一个归一化后的类别分布。严格地说, 它不是模型“天生知道真实概率”, 而是我们设计出来、便于训练和解释的输出形式。

Softmax 还有一个很重要的性质: 它只在乎 logits 的相对差距, 不在乎整体平移。若对所有 logits 同时加上同一个常数 c ,

$$\frac{\exp(y_i + c)}{\sum_j \exp(y_j + c)} = \frac{\exp(c) \exp(y_i)}{\exp(c) \sum_j \exp(y_j)} = \frac{\exp(y_i)}{\sum_j \exp(y_j)}.$$

因此 $(3, 1, -3)$ 与 $(103, 101, 97)$ 的 softmax 输出完全相同。工程实现中常利用这个性质做数值稳定化: 先减去最大 logit $m = \max_j y_j$, 再计算

$$y'_i = \frac{\exp(y_i - m)}{\sum_j \exp(y_j - m)}.$$

³ 视频画面与字幕时间区间: 06:56–07:40。

这样可以避免 $\exp(y_i)$ 因为 logit 太大而溢出。这一点视频里没有展开，但它解释了为什么深度学习框架常把 softmax 与 cross-entropy 合并实现：不仅公式更简洁，数值上也更稳。

深度学习入门学习网 bilibili

Soft-max
$$y'_i = \frac{\exp(y_i)}{\sum_j \exp(y_i)} \quad \begin{array}{l} \blacksquare 1 > y'_i > 0 \\ \blacksquare \sum_i y'_i = 1 \end{array}$$

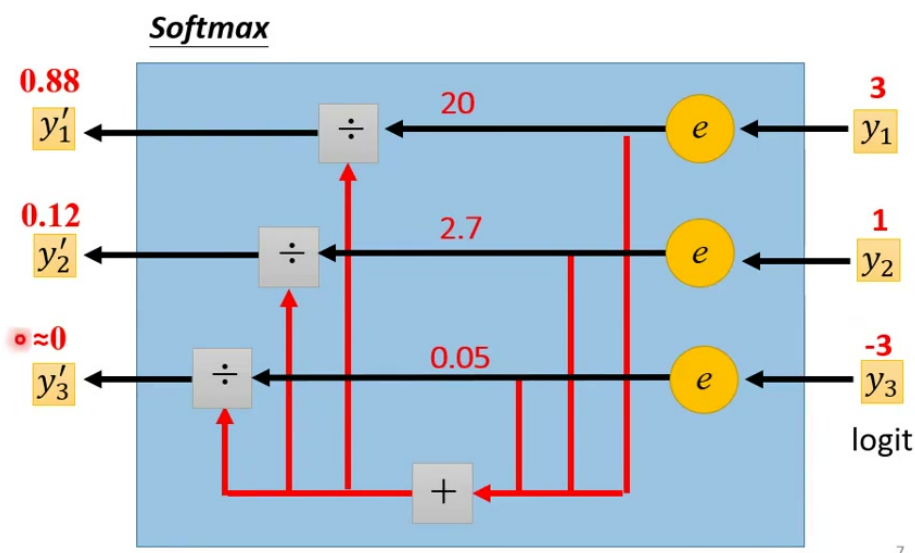


图 4: softmax 的数值例子：若 logits 为 (3, 1, -3)，取 exponential 后约为 (20, 2.7, 0.05)，归一化后得到 (0.88, 0.12, ≈ 0)。softmax 不只把值压到 0 到 1，也会放大大 logit 与小 logit 的差距。⁴

二分类里的 sigmoid 与 softmax

如果只有两个类别，可以直接用二类 softmax；实际工程中更常见的是用 sigmoid 表示二分类。两者并不是互相竞争的不同哲学，而是在二分类设定下可以互相转写的等价形式。多分类时，softmax 是更自然的统一写法。

logit 的角色：先比较，再归一化

softmax 的输入 y_i 常被称作 logit。logit 可以很大、很小、为正、为负，它本身不需要像概率一样被限制在 $[0, 1]$ 。softmax 先把每个 logit 指数化，再用总和归一化，因此真正影响类别结果的是不同 logits 之间的相对大小。如果某个 logit 比其他 logit 大很多，softmax 会把它推到接近 1，其他类别则接近 0。

不要把 softmax 当成“让模型正确”的魔法

Softmax 只负责把 logits 转成一个归一化分布；它不会凭空让模型学会分类。若 logits 本来就把错误类别打得很高，softmax 只会把这个错误变成“很有信心的错误”。真正让 logits 往正确方向移动的，是后面的 loss function 与反向传播。

⁴视频画面与字幕时间区间：09:26–10:08。

4 分类损失：MSE 与 Cross-entropy

得到 $\mathbf{y}'$ 之后，就要定义它和正确标签 $\hat{\mathbf{y}}$ 的距离。一个样本的误差记为 e_n ，整体 loss 是所有样本误差的平均：

$$L = \frac{1}{N} \sum_n e_n.$$

问题在于： e_n 应该怎么定义？本讲比较了两个候选：

$$e_{\text{MSE}} = \sum_i (\hat{y}_i - y'_i)^2,$$

$$e_{\text{CE}} = - \sum_i \hat{y}_i \ln y'_i.$$

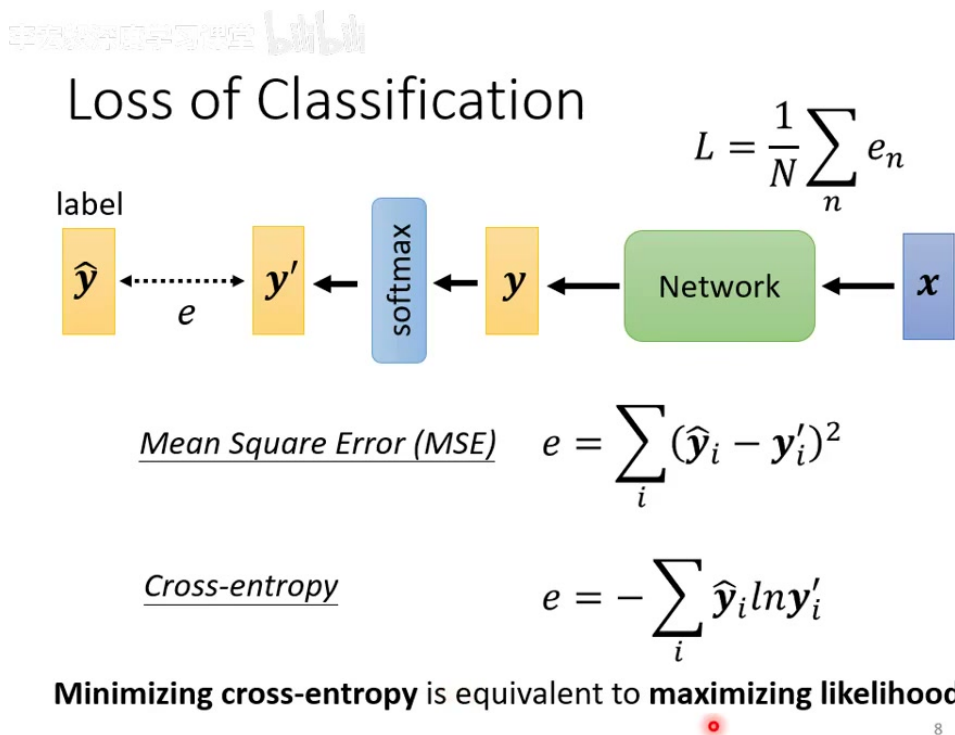


图 5: 分类损失的两个候选：Mean Square Error 直接计算 one-hot 标签与 softmax 输出的平方距离；Cross-entropy 使用 $-\sum_i \hat{y}_i \ln y'_i$ 。在分类任务中，cross-entropy 是更常用的选择；从传统统计观点看，最小化 cross-entropy 等价于最大化 likelihood。⁵

若 $\hat{\mathbf{y}}$ 是 one-hot，假设正确类别为 k ，则 cross-entropy 会简化为

$$e_{\text{CE}} = -\ln y'_k.$$

也就是说，它只关心模型分给正确类别的 softmax 输出 y'_k ：若 y'_k 越接近 1，loss 越小；若 y'_k 越接近 0，loss 会急剧变大。

⁵ 视频画面与字幕时间区间：12:06–13:38。

4.1 从 likelihood 看 cross-entropy

如果把 softmax 输出 y'_i 当作模型给第 i 类的预测概率，那么一个正确类别为 k 的样本，其 likelihood 就是模型分给正确类别的概率：

$$P(\text{correct class} = k \mid \mathbf{x}) = y'_k.$$

最大化 likelihood 等价于最大化 $\ln y'_k$ ，也等价于最小化 $-\ln y'_k$ 。而 one-hot 形式的 cross-entropy

$$-\sum_i \hat{y}_i \ln y'_i$$

在 $\hat{y}_k = 1$ 、其他维度为 0 时正好退化成 $-\ln y'_k$ 。这就是老师在投影片中说的：**minimizing cross-entropy is equivalent to maximizing likelihood**。它不是“两个很像的目标”，而是同一个目标的两种写法。

为什么 PyTorch 里常常看不到 softmax

在 PyTorch 中，CrossEntropyLoss 通常把 log-softmax 与 negative log-likelihood 合在一起实现。因此模型最后一层一般直接输出 logits，不需要手动加 softmax；如果先在网络里加 softmax，再把结果送进 CrossEntropyLoss，就相当于把 softmax 的角色重复处理了一次。

更具体地说，PyTorch 的常见写法是：

```
1 logits = model(x)                # shape: [batch_size, num_classes]
2 loss = criterion(logits, y)      # y: class index, shape [batch_size]
```

其中 y 通常不是 one-hot 向量，而是类别编号，例如 0, 1, 2。这看起来和前面说的 one-hot 不同，其实只是 API 选择：框架内部知道正确类别是哪一维，就能等价地计算 $-\ln p_k$ ，没有必要真的在内存里展开成 one-hot。

一个常见实作错误：重复 softmax

如果模型最后一层已经写了 softmax，又使用 CrossEntropyLoss，就容易把 logits 先压成概率，再交给一个期待 logits 的 loss。这样不仅语义不清，也可能让梯度与数值稳定性变差。多分类训练时，最稳妥的习惯是：**模型输出 logits，loss 使用 cross-entropy；需要显示概率时，再在推理或评估阶段额外 softmax。**

MSE 和 Cross-entropy 的共同目标

如果训练成功，MSE 和 cross-entropy 都希望 $\mathbf{y}'$ 接近 $\hat{\mathbf{y}}$ 。差别不在最终想要的答案，而在**通往答案的 error surface 长什么样**。这正是本讲标题的含义：loss function 的定义会改变 optimization 的难度。

5 为什么 Cross-entropy 更适合分类优化？

老师用一个三分类例子说明 MSE 与 cross-entropy 的差异。设正确标签为

$$\hat{\mathbf{y}} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix},$$

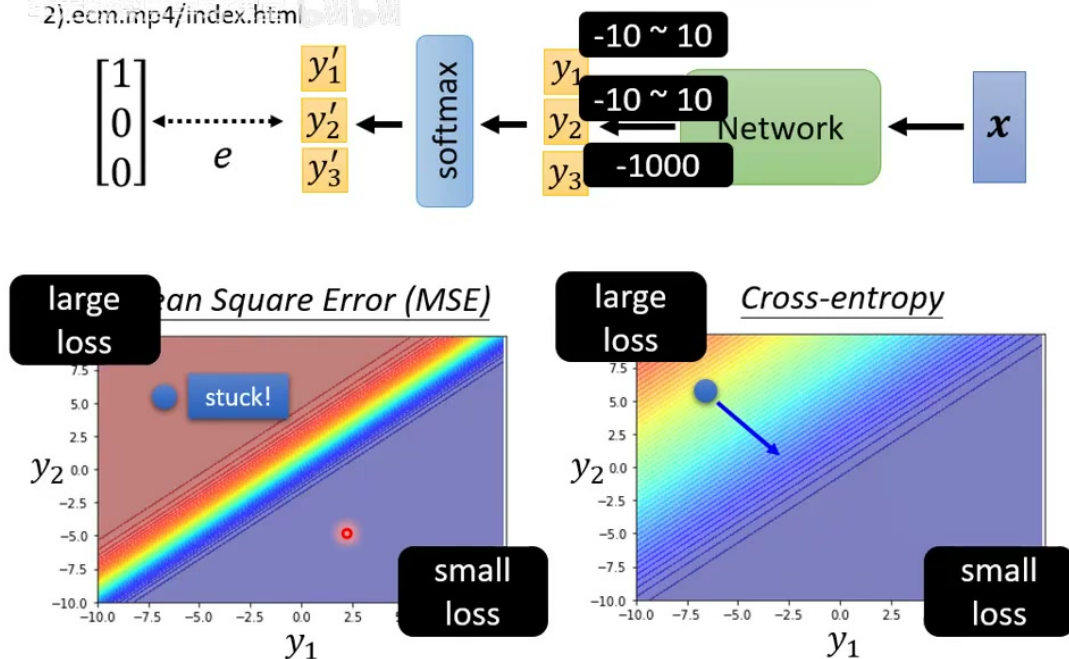
网络输出三个 logits y_1, y_2, y_3 ，通过 softmax 得到 y'_1, y'_2, y'_3 。为了画出二维图，固定

$$y_3 = -1000,$$

只让 y_1 和 y_2 在 $[-10, 10]$ 中变化。因为 y_3 极小，softmax 后 $y'_3 \approx 0$ ，因此主要看 y_1, y_2 如何影响 loss。

当 y_1 很大、 y_2 很小时，正确类别 class 1 的 softmax 输出 y'_1 接近 1，loss 很小；当 y_1 很小、 y_2 很大时，模型把错误类别看得很有信心，loss 很大。两种 loss 都满足这个大方向，但它们的坡度完全不同。

[http://speech.ee.ntu.edu.tw/~tlkagk/courses/MLDS_2015_2/Lecture/Deep%20More%20\(v2\).com.mp4/index.html](http://speech.ee.ntu.edu.tw/~tlkagk/courses/MLDS_2015_2/Lecture/Deep%20More%20(v2).com.mp4/index.html)



Changing the loss function can change the difficulty of optimization.

图 6: MSE 与 cross-entropy 的 error surface 对比。红色区域表示大 loss，蓝色区域表示小 loss。若初始化在左上角这种“错得很有信心”的位置，MSE 的大 loss 区域反而非常平坦，gradient 很小，容易 stuck；cross-entropy 在同一区域仍有明显坡度，可以沿 gradient 往右下角的小 loss 区移动。⁶

⁶ 视频画面与字幕时间区间：18:56–19:23。

MSE 的危险：错得很离谱时，gradient 反而可能很小

MSE 接在 softmax 后面时，若模型在错误类别上已经非常有信心，softmax 容易进入饱和区域。此时输出看起来离目标很远，loss 很大，但对 logits 的梯度可能很小，导致训练起步困难。也就是说，模型不是因为答案接近正确而不动，而是因为错误的 loss surface 让它在糟糕区域也走不动。

Cross-entropy 的优势

Cross-entropy 对“把正确类别预测成接近 0”这件事惩罚非常强： $-\ln y'_k$ 会在 $y'_k \rightarrow 0$ 时迅速变大。它因此能在错得很离谱的区域保留有用的梯度，帮助模型从错误且自信的初始状态中离开。

5.1 把梯度写出来：差别为什么会这么大？

上面的图给的是直觉；再把关键梯度写出来，就能看见为什么 cross-entropy 更顺。记 softmax 输出为

$$p_i = y'_i = \frac{\exp(y_i)}{\sum_j \exp(y_j)}, \quad q_i = \hat{y}_i.$$

其中 $\mathbf{p}$ 是模型预测分布， $\mathbf{q}$ 是 one-hot 标签。对于 softmax + cross-entropy,

$$e_{\text{CE}} = - \sum_i q_i \ln p_i,$$

它对 logit y_i 的梯度会漂亮地化简成

$$\frac{\partial e_{\text{CE}}}{\partial y_i} = p_i - q_i.$$

这个化简可以用一行链式法则看出来。Softmax 的导数为

$$\frac{\partial p_i}{\partial y_\ell} = p_i(\delta_{i\ell} - p_\ell),$$

于是

$$\begin{aligned} \frac{\partial e_{\text{CE}}}{\partial y_\ell} &= \sum_i \frac{\partial e_{\text{CE}}}{\partial p_i} \frac{\partial p_i}{\partial y_\ell} \\ &= \sum_i \left(-\frac{q_i}{p_i} \right) p_i(\delta_{i\ell} - p_\ell) \\ &= -q_\ell + p_\ell \sum_i q_i \\ &= p_\ell - q_\ell, \end{aligned}$$

其中最后一步用到 $\mathbf{q}$ 是 one-hot，因此 $\sum_i q_i = 1$ 。这个结果之所以漂亮，是因为 cross-entropy 里的 $\ln p_i$ 和 softmax 的导数正好互相配合，把复杂的 Jacobian 化成了“预测减标签”。

这条式子信息量很大：如果正确类 k 的预测 p_k 很小，那么 $p_k - q_k \approx -1$ ，梯度仍然很强；如果某个错误类 r 被预测得很大， $p_r - q_r \approx p_r$ ，梯度会明确要求把它压下去。也就是说，模型错得越自信，cross-entropy 给出的纠偏信号越清楚。

MSE 接在 softmax 后面时，情况就不一样。MSE 是

$$e_{\text{MSE}} = \sum_i (q_i - p_i)^2.$$

对 logit 求导时，必须再乘上 softmax 的导数：

$$\frac{\partial p_i}{\partial y_\ell} = p_i(\delta_{i\ell} - p_\ell).$$

因此梯度中会出现 $p_i(1 - p_i)$ 或 $p_i p_\ell$ 这样的因子。当 softmax 已经饱和时，某些 p_i 接近 0 或 1，这些因子就会非常小。于是会出现很尴尬的局面：**预测明明错得很严重，MSE 本身也很大，但传回 logits 的 gradient 却被 softmax 饱和压扁了**。这就是图中 MSE 左上角大 loss 区域却几乎平坦的原因。

举一个极端但很有说明力的例子。假设正确类别是 class 1，但模型 logits 给出

$$\mathbf{y} = (-10, 10, -1000).$$

Softmax 后几乎是

$$\mathbf{p} \approx (0, 1, 0), \quad \mathbf{q} = (1, 0, 0).$$

这代表模型非常自信地选错了 class 2。对 cross-entropy 而言，正确类梯度约为 $p_1 - q_1 \approx -1$ ，错误高分类梯度约为 $p_2 - q_2 \approx 1$ ，所以优化器会明确地提高 y_1 、降低 y_2 。但对 MSE 而言，softmax 已经在 $(0, 1, 0)$ 附近饱和，softmax Jacobian 很小，梯度信号会被削弱；这就是“错得很严重，却不好改”的核心。

最关键的公式对照

对分类任务而言，**softmax + cross-entropy** 的梯度核心是

$$\frac{\partial e}{\partial y_i} = p_i - q_i,$$

它直接比较“预测分布”与“正确 one-hot 标签”。而 **softmax + MSE** 的梯度还要额外乘上 softmax Jacobian，容易在饱和区变小。前者给 optimizer 的路标清楚，后者可能把路标藏在平坦的大 loss 区里。

这不是说 MSE 在数学上“不能”用于分类；在更强的 optimizer、良好的初始化或特定任务中，它未必完全失败。老师也提到，若使用 Adam 这类自适应优化器，gradient 小时学习率可能自动被调大，训练仍有机会继续推进。但这会让训练更困难、更慢，尤其在初期 logits 已经把类别分错且 softmax 饱和时。实际分类任务中，**softmax + cross-entropy** 成为标准搭配，正是因为它给 optimization 更友善的地形。

6 本讲综合：loss function 也是优化设计的一部分

这一讲的核心不是背一个公式，而是建立一个更工程化的判断：**模型训练不起来，不一定只怪模型结构、learning rate 或 optimizer；loss function 本身也可能把 error surface 变得难走**。

分类任务的一条常用路线可以总结为：

1. 用 one-hot vector 表示类别，避免给无序类别强加数轴距离。
2. 让网络最后一层输出与类别数相同的 logits。
3. 用 softmax 把 logits 变成归一化输出。
4. 用 cross-entropy 衡量输出与 one-hot 标签的差距。
5. 在 PyTorch 等框架中，通常让模型输出 logits，直接交给内建的 cross-entropy loss。

从优化角度看，MSE 与 cross-entropy 的差别在于：它们虽然可能有相同的理想终点，却提供了不同的训练地形。Cross-entropy 在模型错得很有信心时仍然给出清楚的下降方向；MSE 则可能在同样区域变得平坦，使 gradient descent 一开始就迈不开步。

一句话总结

改变 loss function 可以改变 optimization 的难度。分类任务选择 cross-entropy，不只是因为它有 likelihood 的解释，更因为它让 gradient descent 面对一个更容易走的 error surface。

和前几讲连起来看

前几讲讨论的是：同一个 loss surface 上，怎样靠 momentum、adaptive learning rate、learning rate scheduling 走得更好。本讲补上另一块拼图：**loss function 本身会决定 surface 长什么样**。所以训练卡住时，除了调 optimizer，也要问一个更上游的问题：我定义的目标函数，是否让正确方向在梯度上足够可见？